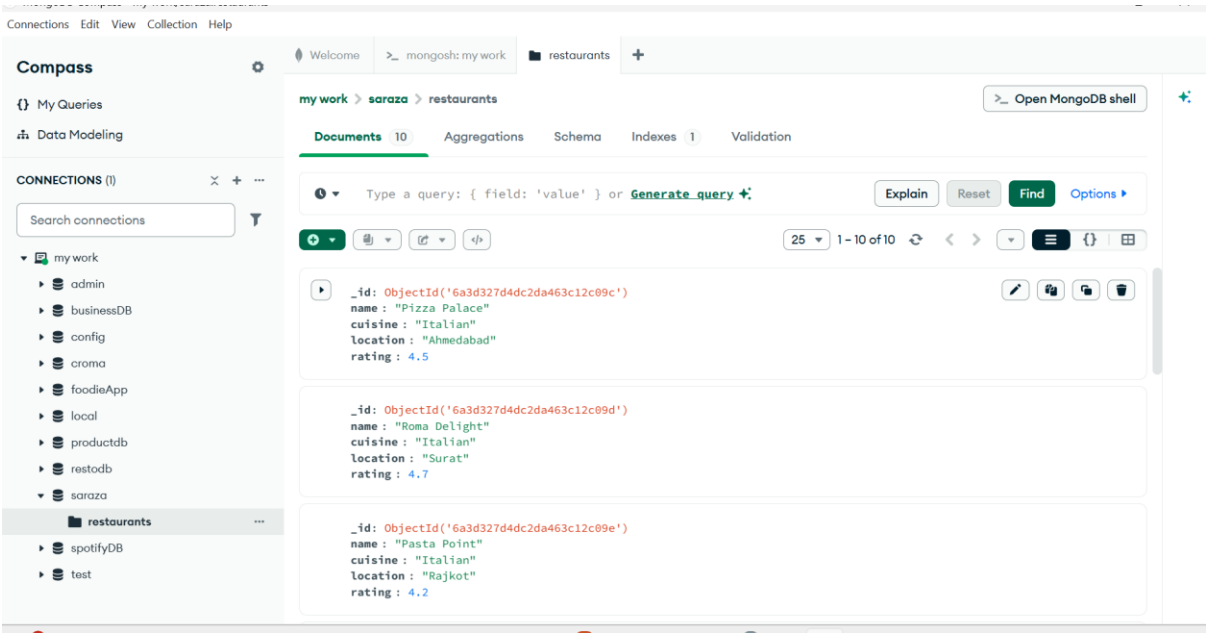
Session 7:

1.

Create a MongoDB collection called 'restaurants' and insert at least 10 documents with fields: name, cuisine, location, and rating. Run a find query to retrieve all restaurants with cuisine 'Italian'.



2.

**Add a single field index on the 'cuisine' field in the 'restaurants' collection using the createIndex() method. Measure and compare the query execution time for finding all 'Italian' restaurants before and after adding the index.

Hint: Use db.restaurants.find({cuisine: 'Italian'}) and db.restaurants.createIndex({cuisine: 1}). Use the .explain('executionStats') method to** check execution time.


```
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'saraza.restaurants',
    indexFilterSet: false,
    parsedQuery: {
      cuisine: {
        '$eq': 'Italian'
      }
    },
    queryHash: 'C08D71C3',
    planCacheKey: 'C08D71C3',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'COLLSCAN',
      filter: {
        cuisine: {
          '$eq': 'Italian'
        }
      },
      direction: 'forward'
```

```
  },
  rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 3,
  executionTimeMillis: 5,
  totalKeysExamined: 0,
  totalDocsExamined: 10,
  executionStages: {
    stage: 'COLLSCAN',
    filter: {
      cuisine: {
        '$eq': 'Italian'
      }
    },
    nReturned: 3,
    executionTimeMillisEstimate: 0,
    works: 11,
    advanced: 3,
    needTime: 7,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    direction: 'forward',
    docsExamined: 10
  }
},
```

```
  command: {
    find: 'restaurants',
    filter: {
      cuisine: 'Italian'
    },
    '$db': 'saraza'
  },
  serverInfo: {
    host: 'LAPTOP-7GI2HGBA',
    port: 27017,
    version: '7.0.37',
    gitVersion: '9d30419d900008ba3ecf2a14546b236f2158b65b'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'forceClassicEngine'
  },
  ok: 1
}



{
```

```
explainVersion: '1',
queryPlanner: {
  namespace: 'saraza.restaurants',
  indexFilterSet: false,
  parsedQuery: {
    cuisine: {
      '$eq': 'Italian'
    }
  },
  queryHash: 'C08D71C3',
  planCacheKey: 'BCAEF92A',
  optimizationTimeMillis: 5,
  maxIndexedOrSolutionsReached: false,
  maxIndexedAndSolutionsReached: false,
  maxScansToExplodeReached: false,
  winningPlan: {
    stage: 'FETCH',
    inputStage: {
      stage: 'IXSCAN',
      keyPattern: {
        cuisine: 1
      },
      indexName: 'cuisine_1',
      isMultiKey: false,
      multiKeyPaths: {
        cuisine: []
      },
      isUnique: false,
      isSparse: false,
```

```
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        cuisine: [
          '["Italian", "Italian"]'
        ]
      }
    }
  },
  rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 3,
  executionTimeMillis: 29,
  totalKeysExamined: 3,
  totalDocsExamined: 3,
  executionStages: {
    stage: 'FETCH',
    nReturned: 3,
    executionTimeMillisEstimate: 29,
    works: 4,
    advanced: 3,
    needTime: 0,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 1,
```

```
docsExamined: 3,
alreadyHasObj: 0,
inputStage: {
 stage: 'IXSCAN',
 nReturned: 3,
 executionTimeMillisEstimate: 29,
 works: 4,
 advanced: 3,
 needTime: 0,
 needYield: 0,
 saveState: 1,
 restoreState: 1,
 isEOF: 1,
 keyPattern: {
  cuisine: 1
 },
 indexName: 'cuisine_1',
 isMultiKey: false,
 multiKeyPaths: {
  cuisine: []
 },
 isUnique: false,
 isSparse: false,
 isPartial: false,
 indexVersion: 2,
 direction: 'forward',
 indexBounds: {
  cuisine: [
   '["Italian", "Italian"]'
```

```
        ]
      },
      keysExamined: 3,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
},
command: {
  find: 'restaurants',
  filter: {
    cuisine: 'Italian'
  },
  '$db': 'saraza'
},
serverInfo: {
  host: 'LAPTOP-7GI2HGBA',
  port: 27017,
  version: '7.0.37',
  gitVersion: '9d30419d900008ba3ecf2a14546b236f2158b65b'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
```

```
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'forceClassicEngine'
  },
  ok: 1
}
```

3.

Create a compound index on the 'cuisine' and 'location' fields in the 'restaurants' collection. Write a query to find all 'Chinese' restaurants in 'Ahmedabad' and check if the compound index is being used by the query planner.

db.restaurants.createIndex({ cuisine: 1, location: 1 })

```
> db.restaurants.createIndex({ cuisine: 1, location: 1 })
< cuisine_1_location_1
> db.restaurants.find({
    cuisine: "Chinese",
    location: "Ahmedabad"
  })
< {
    _id: ObjectId('6a3d327d4dc2da463c12c09f'),
    name: 'Dragon Wok',
    cuisine: 'Chinese',
    location: 'Ahmedabad',
    rating: 4.3
  }
```

```
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          cuisine: 1,
          location: 1
        },
        indexName: 'cuisine_1_location_1',
        isMultiKey: false,
        multiKeyPaths: {
          cuisine: [],
          location: []
        }
```

4.

Drop the index you created on the 'cuisine' field and observe how the query performance changes when searching for a specific cuisine. Record your observations in 2-3 lines.

db.restaurants.dropIndex({ cuisine: 1 })

5.

Use ChatGPT to generate a MongoDB query that would efficiently fetch the top 5 highest rated 'Pizza' restaurants in 'Surat', making sure to use any relevant indexes. Paste the generated query and explain in one line how indexing helps in this scenario.

```
db.restaurants.createIndex({
  cuisine: 1,
  location: 1,
  rating: -1
})
cuisine_1_location_1_rating_-1
db.restaurants.find({
  cuisine: "Pizza",
  location: "Surat"
})
.sort({ rating: -1 })
.limit(5)
{
  _id: ObjectId('6a3d327d4dc2da463c12c0a3'),
  name: 'Pizza Hub',
  cuisine: 'Pizza',
  location: 'Surat',
```